

오늘의 작업기록 — 2026년 6월 22일

mini-ledger 증분 2: `getAccount` (+ 내 코드 vs 정답 코드 전면 비교)

한눈에

항목	내용
목표	증분 2 마무리 (<code>getAccount</code> + <code>transfer</code> + <code>test Main</code>)
실제 완료	<code>getAccount</code> ✓ (컴파일 통과 → 커밋 → 푸시)
미완 → 다음	<code>transfer</code> , <code>test Main</code>
오늘의 핵심	<code>getAccount</code> 의 막힌 4곳을 정확히 해부 + 스마트 구두점 함정 영구 해결
새 개념	<code>Map.get</code> 는 없으면 예외가 아니라 <code>null</code> 반환 / 반환 타입과 <code>return</code> / <code>== null</code> 검사
저장소	github.com/Taewoo-Won/mini-ledger
상태	제로베이스 초반 — 가장 더딘 구간 통과 중

오늘은 솔직히 힘들었다. 빈칸에서 막혀 정답을 받았다. 하지만 "어디서, 왜 막혔는지"를 4곳 모두 해부했다. 그게 이 기록의 핵심이다.

0. 오늘의 출발점 (6/21 이후)

- 어제(6/21)까지: `Bank.java` 에 `createAccount` 까지 완성 + 커밋 `e02b204` .
- 오늘 할 일: 같은 파일에 `getAccount` 추가 → (그다음 `transfer` → `test Main`).
- 읽은 책: 점프 투 자바 75-89p (03-07 리스트, 03-08 맵) → `Map` 의 `get` / `put` 습득.

1. `getAccount` 는 무엇을 만드는가

`getAccount(id)` = `id`로 계좌를 찾아 돌려준다. 없으면 예외.

3단계 흐름:

꺼낸다(`get`) → 없으면 막는다(`null` 이면 `throw`) → 있으면 돌려준다(`return`)

`createAccount` (넣기)의 짝 — 꺼내기다.

오늘 새로 배운 핵심 사실 — `Map.get` + `null`

`accounts.get(id)` 는 키가 없을 때 예외를 던지지 않는다. `null` 을 돌려준다.

→ "없으면 알아서 터지겠지"는 틀림. 내가 직접 `null` 을 잡아서 `throw`해야 한다. (이게 오늘의 함정 1)

2. ★ 내 코드 vs 정답 코드 — 전면 비교 ★

2-1. 내가 짰 `getAccount` (원문 그대로)

```
public Account getAccount(String id) {
    Account account = accounts.get( __A__ );
    if ( __B__ ) {
        throw new IllegalArgumentException("계좌를 찾을 수 없습니다");
    }
    return __D__;
}
```

2-2. 정답 `getAccount` (원문 그대로)

```
public Account getAccount(String id) {
    Account account = accounts.get(id);
    if (account == null) {
        throw new IllegalArgumentException("계좌를 찾을 수 없습니다");
    }
    return account;
}
```

2-3. 줄별 차이 — 정확히 4곳

#	위치	내가 쓴 것	정답	무엇이 / 왜
1	<code>get</code> 인자	<code>accounts.get(__A__)</code>	<code>accounts.get(id)</code>	A 못 채움. 맵에서 꺼낼 키=받은 파라미터 <code>id</code> . <code>createAccount</code> 에서 <code>put(id, ...)</code> 로 넣었으니, 꺼낼 때도 <code>id</code> .
2	<code>if</code> 조건	<code>if (__B__)</code>	<code>if (account == null)</code>	B 못 채움. <code>get</code> 가 못 찾으면 <code>null</code> 반환 → <code>null</code> 이면 "계좌 없음". <code>account == null</code> 이 바로 그 검사.
3	따옴표	<code>("계좌를...")</code> 둥근	<code>("계좌를...")</code> 곧은	환경 합정 . 폰 스마트 구두점이 <code>"</code> 를 <code>“ ”</code> 로 자동 변환. 자바는 둥근 따옴표를 문자열로 못 읽음 → 컴파일 에러 .
4	<code>return</code>	<code>return __D__;</code>	<code>return account;</code>	D 못 채움. 위에서 담은 그 계좌(<code>account</code>)를 반환.

2-4. 핵심 진단 (이게 제일 중요)

4곳 중 — **3곳 (A · B · D)** = *"뭘 타이핑할지 몰랐던 것."* 논리를 틀린 게 아니라 문법이 아직 손에 안 붙은 것. 개념(키= `id` , 없으면= `null` , 반환= `account`)만 잡으면 채워진 다. - **1곳 (따옴표)** = 코딩 실력과 무관한 환경 문제. 키보드가 글자를 바꾼 것.

즉, 오늘 막힌 건 "내가 멍청해서"가 아니라 **(1) 초반이라 문법 미숙 + (2) 폰의 자동 변환**. 둘 다 시간·설정으로 풀리는 것이다.

2-5. 전체 파일 비교 (`Bank.java` 통째로)

Before — 내 버전 (getAccount 깨짐):

```
import java.util.Map;
import java.util.HashMap;

public class Bank {
    private Map<String, Account> accounts = new HashMap<>();

    public void createAccount(String id, String owner, long initialBalance) {
        Account account = new Account(id, owner, initialBalance);
        accounts.put(id, account);
    }

    public Account getAccount(String id) {
        Account account = accounts.get( __A__ );
        if ( __B__ ) {
            throw new IllegalArgumentException("계좌를 찾을 수 없습니다");
        }
        return __D__;
    }
}
```

After — 정답 (완성):

```
import java.util.Map;
import java.util.HashMap;

public class Bank {
    private Map<String, Account> accounts = new HashMap<>();

    public void createAccount(String id, String owner, long initialBalance) {
        Account account = new Account(id, owner, initialBalance);
        accounts.put(id, account);
    }

    public Account getAccount(String id) {
        Account account = accounts.get(id);
        if (account == null) {
            throw new IllegalArgumentException("계좌를 찾을 수 없습니다");
        }
        return account;
    }
}
```

추가 차이: **들여쓰기**. 내 버전은 `getAccount` 가 `createAccount` 와 정렬이 안 맞고, 끝에 빈 줄·중괄호 간격이 들쭉날쭉했다. 자바는 공백을 무시하므로 **에러는 아니지만**, 정렬하면 읽기 쉽다. (실무에선 정렬이 곧 기본기다.)

3. 디버깅 케이스: 스마트 구두점 합정

- 증상: “계좌를 찾을 수 없습니다” — 따옴표가 동글다.
- 진단: 자바 문자열은 “곧은 따옴표” 만 인식. “등근 따옴표” 는 다른 유니코드 문자 → 컴파일 에러.
- 근본 원인: iPhone "스마트 구두점" 기능이 " " 입력을 자동으로 등근 따옴표로 변환.
- 해결: 설정 → 일반 → 키보드 → 스마트 구두점 끄기. 한 번 끄면 영구 해결.
- 교훈: 환경이 코드를 망칠 수 있다. 코드 논리가 맞는데 에러가 나면 → 눈에 안 보이는 것(따옴표 종류, 공백, 숨은 문자)을 의심하라. 이런 앞으로도 계속 씨먹을 디버깅 본능이다.

4. 산출물 (깃허브 올리기까지)

- 파일: ~/mini-ledger/Bank.java — getAccount 추가 완료
- 컴파일: bash cd ~/mini-ledger # 작업 전 폴더부터 (6/21 그 교훈) javac *.java # 에러 0 확인
- 커밋 · 푸시: bash git add . && git commit -m "implement getAccount" && git push
- 커밋 메시지: implement getAccount
- 커밋 해시는 git log 에서 확인 (참고 — 이전: Account = 9e671d6 , Bank createAccount = e02b204)
- 푸시 완료 = GitHub 반영 + 잔디.
- 저장소: github.com/Taewoo-Won/mini-ledger

5. 다음 (바로 이어서)

1. transfer — 내가 직접 쓴다. 도구는 다 있다: getAccount ×2 + withdraw + deposit . - 안전한 순서: 두 계좌 먼저 가져오기(없으면 throw) → 출금 → 입금. 그래야 돈이 안 증발한다. - 위임: amount <= 0 재검사 안 함(withdraw / deposit 이 이미 한다).
2. test Main — 3케이스로 굴러 증명: - ① 정상 이체 (두 잔액 출력) - ② 없는 계좌 이체 (예외 + from 잔액 안 줄었나 ← 돈 증발 방지, 핵심) - ③ 잔액 부족 이체 (예외)
3. 이 둘 끝나면 = 증분 2 잔짜 완료.

6. 개념 사전

- 반환 타입 (return type): 메서드가 돌려주는 값의 종류. getAccount 는 void 가 아니라 Account 를 돌려줌 → 끝에 return account; 필요. (getBalance 가 long 돌려주던 것과 같은 원리. deposit 은 void 라 return 없음.)
- null : "아무것도 없음" 신호. accounts.get(id) 는 키가 없으면 예외가 아니라 null 반환 → 직접 null 검사 필수.
- == null 검사: account == null 은 "account가 비었나?"를 묻는 것. 참이면 계좌 없음 → throw.
- 곧은 따옴표 vs 등근 따옴표: " " (자바 OK) vs “ ” (자바 X). 겉보기 비슷하지만 다른 문자.
- 객체.메서드 호출: accounts.get(id) , accounts.put(...) 처럼 어떤 객체에게 동작을 시키는 형태. (transfer에서 쓸 from.withdraw(amount) 도 같은 꼴.)
- (예고) 위임: 한 번 검사한 건 또 검사 안 함 — transfer가 amount <= 0 을 재검사 안 하는 이유. (어제 Bank가 this.id 안 한 것과 같은 이치.)

7. 비유 모음

- Map.get + null : 사물함을 열었는데 비어 있으면 "없음"(null) 표시가 뜨는 것. 사물함이 폭발(예외)하진 않는다 — 그래서 내가 "비었네?"를 직접 확인해야 한다.
- getAccount : 전화번호부에서 이름으로 찾기 — 있으면 번호 반환, 없으면 "그런 사람 없음" 알림(throw).
- 곧은/등근 따옴표: 겉모습 똑같은 쌍둥이인데, 자바는 한 명만 알아본다. 다른 쌍둥이를 보내면 "누구세요?"(에러).

8. 회고

오늘은 솔직히 힘든 날이었다. getAccount 빈칸에서 막혀 결국 정답을 받았다. 근데 그게 후퇴는 아니다 — 막힌 4곳이 정확히 뭐였는지 이제 안다. 3곳은 문법 미숙(개념 잡으면 끝), 1곳은 따옴표 합정(설정으로 영구 해결). "왜 막혔는지"를 아는 것과 그냥 베끼는 것은 다르다.

제로베이스 초반은 원래 제일 더디고 아픈 구간이다 — 문법이 손에 붙는 단계라서. 오늘 속도가 앞으로의 속도가 아니다. 지금 통제할 건 꾸준함과 환경(스마트 구두점도 이제 껐다)이지, 4일째 타이핑 속도가 아니다.

다음엔 transfer를 내 손으로 쓴다. 도구는 이미 다 가졌다.